TI102 - Algorithmique

TD7 - Matrices et Tableaux 2D - Fonctions

Consignes:
- Le travail est **individuel**.
- Le travail est à faire sur papier.
- Il est fortement recommandé de se référer aux cours et aux supports de cours pour répondre aux questions.

Acquis d'apprentissage:
À la fin de ce TD, vous serez capable de:
- Comprendre le fonctionnement d'un tableau 2D.
- Écrire les boucles nécessaires pour parcourir un tableau 2D.
- Résoudre des problèmes simples faisant intervenir des tableaux 2D.
- Écrire le prototype d'une fonction selon sa description.
- Définir et appeler des fonctions lors de la résolution de problème.
- Transformer les algorithmes des problèmes des TD précédents en fonctions.
- Résoudre des problèmes complexes en les décomposant en sous-problèmes plus simples.

Niveau débutant

`Échauffement sur les tableaux 2D`

Exercice 1. Donnez la sortie des algorithmes suivants:

```
Algorithme : algo1
Variables :
  mat[20][20] : Tableau d'entiers
  i, j : entier
  li, co : entier
Début
  1: mat ← {{12, 34, 5}, {2, −7, 23}, {10, 4, −8}}
  2: li ← 3
  3: co ← 3
  4: Pour i ← 1 à li Faire
  5:   Pour j ← 1 à co Faire
  6:       Ecrire(mat[i][j], " ")
  7:   Fin Pour
  8: Fin Pour
Fin
```

```
Algorithme : algo2
Variables :
  mat[20][20] : Tableau d'entiers
  i, j : entier
  li, co : entier
Début
  1: mat ← {{12, 34, 5}, {2, −7, 23}, {10, 4, −8}}
  2: li ← 3
  3: co ← 3
  4: Pour i ← 1 à li Faire
  5:   Pour j ← 1 à co Faire
  6:       Ecrire(mat[i][j], " ")
  7:   Fin Pour
  8:   Ecrire("\NL")
  9: Fin Pour
Fin
```

```
Algorithme : algo3
Variables :
  mat[20][20] : Tableau d'entiers
  i, j : entier
  li, co : entier
Début
  1: mat ← {{12, 34, 5}, {2, −7, 23}, {10, 4, −8}}
  2: li ← 3
  3: co ← 3
  4: Pour i ← 1 à li Faire
  6:    Ecrire(mat[i][i], " ")
  7: Fin Pour
Fin
```

Exercice 2. Repérez les erreurs dans les algorithmes suivants et corrigez-les.

```
Algorithme : algo4
Variables :
  mat[20][20] : Tableau d'entiers
  i, j : entier
  li, co : entier
Début
  1: mat ← {{12, 34, 5}, {2, −7, 23}, {10, 4, −8}}
  2: li ← 3
  3: co ← 3
  4: Pour i ← 1 à li Faire
  5:    Ecrire(mat[i])
  6: Fin Pour
Fin
```

```
Algorithme : algo5
Variables :
  mat[2][3] : Tableau d'entiers
  i, j : entier
  li, co : entier
Début
  1: li ← 2
  3: co ← 3
  4: Pour i ← 1 à li Faire
  5:    Pour j ← 1 à co Faire
  6:          Lire(mat[j][i])
  7:    Fin Pour
  8: Fin Pour
Fin
```

```
Algorithme : algo6
Variables :
  mat[20][20] : Tableau d'entiers
  i, j : entier
  li, co : entier
Début
  1: mat ← {{12, 34, 5}, {2, −7, 23}}
  2: li ← 2
  3: co ← 3
  4: Pour i ← 1 à li Faire
  5:    Pour j ← 1 à co Faire
  6:          Ecrire(mat[j][i])
  7:    Fin Pour
  8: Fin Pour
Fin
```

`Échauffement sur les fonctions`

Exercice 3. Pour chacune des descriptions de fonctions suivantes, donnez leur prototype. (Indication: posez-vous la question sur la modification ou non des paramètres d'entrée.)

1. Une fonction qui calcule le produit de deux entiers.
2. Une fonction qui calcule la somme des éléments d'un tableau d'entiers.
3. Une fonction qui crée, puis rempli et renvoie un tableau d'entiers de taille logique n par une saisie utilisateur.

4. Une fonction qui affiche un tableau d'entiers.
5. Une fonction qui converti tous les caractères d'une chaîne de caractères en majuscules.
6. Une fonction qui détermine si une chaîne de caractères est une sous-chaîne d'une autre chaîne de caractère.
7. Une fonction qui calcule le déterminant d'une matrice carrée d'entiers.

Exercice 4. Étant donnés les deux prototypes de fonctions suivantes, repérez et corrigez les erreurs glissées dans les algorithmes ci-dessous.

- **Fonction:** $f(a$: entier, b: réel)
- **Fonction:** $g(a$: Tableau de réel, b: réel, c: entier) $\rightarrow$ booléen

```
Algorithme : algo7
Variables :
  x, y : entier
Début
  x ← 5
  y ← f(x, 0.7)
Fin
```

```
Algorithme : algo8
Variables :
  x : réel
Début
  x ← 0.7
  f(x, x)
Fin
```

```
Algorithme : algo9
Variables :
  x : réel
Début
  f(5, x)
Fin
```

```
Algorithme : algo10
Variables :
  x : réel
  tab[10] : Tableau de réel
  ans : booléen
Début
  ans ← g(5, 0.5, {1.0, 13.8})
Fin
```

Niveau intermédiaire

Exercice 5. Écrire une fonction affiche_mat qui prend en entrée une matrice de taille $n \times m$, ainsi que les tailles logiques n et m et qui affiche tous les éléments de la matrice, tous les éléments appartenant à la même ligne étant séparés par un espace et chaque ligne étant séparée par un retour à la ligne.

Exemple : *L'appel de affiche_mat*$(\{\{1,2,3\},\{4,5,6\}\},2,3)$ *doit afficher:*

```
1 2 3
4 5 6
```

Exemple : *Et l'appel de affiche_mat*$(\{\{1,0,0\},\{0,1,0\},\{0,0,1\}\},3,3)$ *doit afficher:*

```
1 0 0
0 1 0
0 0 1
```

Exercice 6. Écrire une fonction somme_mat qui prend en entrée une matrice de taille $n \times m$, ainsi que les tailles logiques n et m et qui renvoie la somme de tous les éléments de la matrice.

Exemple : *L'appel de somme_mat*$(\{\{1,2,3\},\{4,5,6\}\},2,3)$ *doit renvoyer 21 car* $1+2+3+4+5+6=21$. *Et l'appel de somme_mat*$(\{\{1,0,0\},\{0,1,0\},\{0,0,1\}\},3,3)$ *doit renvoyer 3.*

Exercice 7. Écrire une fonction somme_diag qui prend en entrée une matrice carrée de taille $n \times n$ et qui renvoie la somme des éléments de la diagonale principale de la matrice.

Exemple : *L'appel de somme_diag*$(\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\},3)$ *doit renvoyer 15 car* $1+5+9=15$. *Et l'appel de somme_diag*$(\{\{1,0,0\},\{0,1,0\},\{0,0,1\}\},3)$ *doit renvoyer 3.*

Exercice 8. Écrire une fonction somme_anti_diag qui prend en entrée une matrice carrée de taille $n \times n$ et qui renvoie la somme des éléments de la diagonale secondaire de la matrice.

Exemple : *L'appel de somme_anti_diag*$(\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\},3)$ *doit renvoyer 15 car* $3+5+7=15$. *Et l'appel de somme_anti_diag*$(\{\{1,0,0\},\{0,1,0\},\{0,0,1\}\},3)$ *doit renvoyer 1.*

Exercice 9. Écrire une fonction somme_ligne qui prend en entrée une matrice de taille $n \times m$, ainsi que les tailles logiques n et m et un entier i et qui renvoie la somme des éléments de la ligne i de la matrice.

Exemple : *L'appel de somme_ligne*$(\{\{1,2,3\},\{4,5,6\}\},2,3,1)$ *doit renvoyer 15 car* $4+5+6=15$.
Et l'appel de somme_ligne$(\{\{1,0,0\},\{0,1,0\},\{0,0,1\}\},3,3,2)$ *doit renvoyer 1.*

Exercice 10. Écrire une fonction somme_colonne qui prend en entrée une matrice de taille $n \times m$, ainsi que les tailles logiques n et m et un entier j et qui renvoie la somme des éléments de la colonne j de la matrice.

Exemple : *L'appel de somme_colonne*$(\{\{1,2,3\},\{4,5,6\}\},2,3,1)$ *doit renvoyer 6 car* $2+5=6$.
Et l'appel de somme_colonne$(\{\{1,0,0\},\{0,1,0\},\{0,0,1\}\},3,3,2)$ *doit renvoyer 1.*

Exercice 11. La transposée d'une matrice M de taille $n \times m$ est une matrice M^T de taille $m \times n$ telle que pour tout $i \in \{1,\dots,n\}$ et $j \in \{1,\dots,m\}$, $M^T[j][i] = M[i][j]$. Écrire une fonction transpose_mat qui prend en entrée une matrice de taille $n \times m$, ainsi que les tailles logiques n et m et qui renvoie la transposée de la matrice.

Exemple : *L'appel de transpose_mat*$(\{\{1,2,3\},\{4,5,6\}\},2,3)$ *doit renvoyer la matrice*

```
1 4
2 5
3 6
```

Exemple : *L'appel de transpose_mat*$(\{\{1,0,0\},\{0,1,0\},\{0,0,1\}\},3,3)$ *doit renvoyer la matrice*

```
1 0 0
0 1 0
0 0 1
```

Niveau avancé

Exercice 12. Le produit de deux matrices A et B de tailles respectives $n \times m$ et $m \times p$ est une matrice C de taille $n \times p$ telle que pour tout $i \in \{1,\dots,n\}$ et $j \in \{1,\dots,p\}$, $C[i][j] = \sum_{k=1}^{m} A[i][k] \times B[k][j]$. Par exemple,

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix} = \begin{pmatrix} 1\times7+2\times9+3\times11 & 1\times8+2\times10+3\times12 \\ 4\times7+5\times9+6\times11 & 4\times8+5\times10+6\times12 \end{pmatrix} = \begin{pmatrix} 58 & 64 \\ 139 & 154 \end{pmatrix}$$

Écrire une fonction produit_mat qui prend en entrée deux matrices A et B de tailles respectives $n \times m$ et $m \times p$, ainsi que les tailles logiques n, m et p et qui renvoie le produit de ces deux matrices.

Exercice 13. Un carré magique est une matrice carrée de taille $n \times n$ telle que la somme de tous les éléments de chaque ligne, de chaque colonne et des deux diagonales est égale. De plus le carré doit contenir une fois chaque nombre, de 1 à n^2. Le but de cet exercice est d'écrire une fonction est_carre_magique qui prend en entrée une matrice carrée de taille $n \times n$ et qui renvoie un booléen indiquant si la matrice est un carré magique ou non.

1. Écrire une fonction aplatir qui prend en entrée une matrice carrée de taille $n \times n$ et qui renvoie un tableau d'entiers contenant tous les éléments de la matrice.

   Exemple : *Appliqué à la matrice* $\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\}$, *la fonction doit renvoyer le tableau* $\{1,2,3,4,5,6,7,8,9\}$.

2. Écrire une fonction domaine qui prend en entrée un tableau d'entiers et sa taille logique n et qui renvoie un Vrai si le tableau contient tous les entiers de 1 à n et renvoie Faux sinon.

   Exemple : *Appliqué au tableau* $\{1,2,3,4,5,6,7,8,9\}$, *la fonction doit renvoyer Vrai. Par contre, appliqué au tableau* $\{1,2,3,4,5,6,7,8,8\}$, *la fonction doit renvoyer Faux.*

3. Écrire une fonction differents qui prend en entrée un tableau d'entiers et sa taille logique n et qui renvoie un Vrai si tous les éléments du tableau sont différents et renvoie Faux sinon.

   Exemple : *Appliqué au tableau* $\{1,2,3,4,5,6,7,8,9\}$, *la fonction doit renvoyer Vrai. Par contre, appliqué au tableau* $\{1,2,3,4,5,6,7,8,8\}$, *la fonction doit renvoyer Faux.*

4. En vous aidant des fonctions précédentes ainsi que des fonctions somme_ligne, somme_colonne, somme_diag et somme_anti_diag, écrire la fonction est_carre_magique qui prend en entrée une matrice M de taille $n \times n$ et qui vérifie si M est magique ou non.

Exercice 14. On considère un robot qui se déplace sur une grille de taille $r \times c$. Le robot commence à la case de coordonnées $(1,1)$ et doit atteindre la case de coordonnées (r,c) en se déplaçant uniquement soit d'une case vers la droite soit d'une case vers le bas à chaque déplacement.

Comme le robot commence toujours à $(1,1)$. Si la grille est de taille 1×1, il n'y a qu'une seule façon d'atteindre $(1,1)$: c'est de rester sur place. Le nombre de façons d'atteindre la case (r,c) est donc 1. Si la grille est de taille 2×2, il y a deux façons d'atteindre la case $(2,2)$: soit en allant à droite puis en bas, soit en allant en bas puis à droite.

1. On considère une grille de taille $r \times c$. On notera par $c(i,j)$ le nombre de façons d'atteindre la case (i,j) à partir de $(1,1)$.
   a) Que vaut $c(1,1)$?
   b) Puis pour tout $1 \leq i \leq r$ et pour tout $1 \leq j \leq c$, que vaut $c(1,j)$ et que vaut $c(i,1)$?

2. Puisque les déplacements ne peuvent se faire que vers la droite ou vers le bas, le robot ne peut donc atteindre la case (i,j) que s'il est passé par la case $(i-1,j)$ ou par la case $(i,j-1)$. Pour tout $1 \leq i \leq r$ et $1 \leq j \leq c$, déduire l'expression de $c(i,j)$ en fonction de $c(i-1,j)$ et $c(i,j-1)$.

3. Écrire une fonction nb_chemins qui prend en entrée le nombre de lignes r et le nombre de colonne c de la grille et qui renvoie le nombre de façons différentes de se rendre de la case $(1,1)$ à la case (r,c). Vous pourrez utiliser une matrice C de taille $r \times c$ pour stocker les valeurs de $c(i,j)$. (La solution est donc l'élément $C[r][c]$).

Exercice 15. Dans cet exercice, il vous est donné une liste de pièces de monnaie et un montant. Le but est de déterminer le minimum de pièces de monnaie qu'il faut pour payer le montant donné. On suppose que l'on a une quantité illimitée de chaque pièce de monnaie.

Par exemple, si on a les pièces de monnaie $\{1,4,5\}$ et que l'on veut payer 13, une manière de faire est de payer $5+5+1+1+1=13$ avec 5 pièces. Une autre manière de faire est de payer $4+4+4+1=13$ avec 4 pièces, *etc.*. Mais la solution optimale est de payer $4+4+5=13$ avec 3 pièces.

Écrire une fonction min_pieces qui prend en entrée une liste de pièces de monnaie et un montant et qui renvoie le minimum de pièces de monnaie qu'il faut pour payer le montant donné.

Exemple : *L'appel de min_pieces*$(\{1,4,5\},13)$ *doit renvoyer* 3.

Exercice 16. La somme des préfixes d'une séquence d'entiers $x_1, x_1, x_2, \ldots$ est une séquence d'entiers $y_1, y_2, \ldots$ telle que $y_1 = x_1, y_2 = x_1 + x_2, y_3 = x_1 + x_2 + x_3, \ldots$.

Dans cet exercice une séquence de n entiers $x_1, x_2, x_3, \ldots, x_n$ sera représentée par un tableau de tab de n éléments.

1. Écrire une fonction somme_prefixe qui prend en paramètres un tableau d'entiers tab et qui renvoie un tableau d'entiers pre qui contient la somme des préfixes de tab. Le prototype de votre fonction doit être la suivante:

   Fonction: somme_prefix (tab: tableau d'entier, n: entier) → tableau d'entier

2. La somme des préfixes d'un tableau d'entiers tab permet de retrouver rapidement la somme des éléments de tab compris entre deux indices donnés. En utilisant la fonction somme_prefixe, écrire une fonction somme_sous_tableau qui prend en paramètres un tableau tab, sa taille n, deux positions i et j dans tab et renvoie la somme des éléments de tab compris entre i et j (inclus). Le prototype de votre fonction doit être la suivante:

   Fonction: somme_sous_tableau (tab: tableau d'entier, n: entier, i: entier, j: entier) → entier

3. De la fonction somme_sous_tableau, on peut déduire une manière de calculer la somme des éléments d'un tableau tab sans parcourir ses éléments. Écrire une fonction somme qui prend en paramètres un tableau tab et sa taille n en utilisant uniquement un appel à la fonction somme_sous_tableau (sans utiliser de boucle Pour).

4. En vous aidant des fonctions des questions précédentes, écrire une fonction indice_equilibre qui prend en paramètres un tableau tab, sa taille n et renvoie la position i dans tab telle que les sommes des éléments de tab à gauche et à droite de i sont égales. Si une telle position n'existe pas, la fonction renvoie -1.

   Exemple : *indice_equilibre*$(\{1,2,1,2,1\},5)$ *renvoie 3 et indice_equilibre*$(\{3,-1,-5,8\},4)$ *renvoie 2; tandis que indice_equilibre*$(\{1,2,3,4\},4)$ *renvoie -1*.